OI 板子

目录

1. OI 模板

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <queue>
#include <stack>
#include <map>
#include <set>
#include <unordered_map>
#include <unordered_set>
#include <bitset>
#include <string>
#include <numeric>
#include <algorithm>
#include <cmath>
#include <climits>
#include <cstring>
using namespace std;
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    return 0;
}
```

2. __int128

```
// C 风格
__int128 read() {
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-')
            f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9') {
        x  = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}
void print(__int128 x) {
    if (x < 0) {
        putchar('-');
        x = -x;
    }
    if (x > 9) {
        print(x / 10);
    }
    putchar(x % 10 + '0');
}
// C++风格
istream& operator>>(istream& is, __int128& a) {
    __int128 x = 0, f = 1;
    char ch;
    is.get(ch);
    while (ch < '0' || ch > '9') {
        if (ch == '-')
            f = -1;
        is.get(ch);
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        is.get(ch);
    }
    a = x * f;
    return is;
}
ostream& operator<<(ostream& os, __int128 x) {
    if (x < 0) {
        os << '-';
        x = -x;
    }
    if (x > 9) {
        os << __int128(x / 10);
    }
    os << char(x % 10 + '0');
    return os;
}
```

3. bit

3.1. qpow(a,x,MOD)

```cpp
int qpow(int a, int b, int MOD) {
    int ret = 1;
    while (b) {
        if (b & 1) {
            ret = ret * a % MOD;
        }
        a = a * a % MOD;
        b >>= 1;
    }
    return ret;
}
```

3.2. lowbit(x)

```cpp
int lowbit(int x) {
    return x & (-x);
}
```

4. 二分

4.1. 库函数

```cpp
equal_range();
lower_bound();
upper_bound();
binary_search();
equal_range=[lower_bound,upper_bound)
binary_search=is_valid(equal_range)
```

4.2. 整数集合上的二分

```cpp
/**
 * @name lower_bound
 * @brief 返回第一个大于等于 x 的下标
 * @note 按照左闭右开的规范
 */
int lower_bound(const vector<int>& a, int l, int r, int x) {
    while (l < r) {
        // mid 取不到 r
        // 下面三句话配套出现
        int mid = (l + r) >> 1;
        if (a[mid] >= x) {
            r = mid;
        }
        else {
            l = mid + 1;
```

```
        }
    }
    // 二分终止条件：l==r
    return l;
}
/**
 * @name
 * @brief 返回最后一个小于等于 x 的下标
 * @note 按照左闭右开的规范
 */
int bound(const vector<int>& a, int l, int r, int x) {
    while (l < r) {
        // mid 取不到 l
        // 下面三句话配套出现
        int mid = (l + r + 1) >> 1;
        if (a[mid] <= x) {
            l = mid;
        }
        else {
            r = mid - 1;
        }
    }
    // 二分终止条件：l==r
    return l;
}
```

```
double bound(double l, double r) {
    while (l + 1e-5 < r) {
        double mid = (l + r) / 2;
        if (calc(mid)) {
            r = mid;
        }
        else {
            l = mid;
        }
    }
    return l;
}
```

4.3. 实数域上的二分

```
bool calc(double x) {
    return x == 114514.1919810;
}
```

5. 排序

5.1. 库函数

5.1.1. 快速

```
sort(iterator begin, iterator end [, __Function cmp]);
```

5.1.2. 稳定

```
stable_sort(iterator begin, iterator end [, __Function cmp]);
```

5.1.3. 前`mid-begin`个都有序

```
partial_sort(iterator begin, iterator mid, iterator end [, __Function cmp]);
```

5.1.4. 第`mid-begin`个位置正确

```
nth_element(iterator begin, iterator nth, iterator end [, __Function cmp]);
```

5.1.5. 判断有序

```
is_sorted(iterator begin, iterator end [, __Function cmp]);
```

5.2. 去重

```
sorted.erase(std::unique(sorted.begin(), sorted.end()), sorted.end());
```

5.3. 离散化

```
vector<int> discrete(vector<int> a) {
    if (a.empty()) {
        return {};
    }
    sort(a.begin(), a.end());
    a.erase(std::unique(a.begin(), a.end()), a.end());
    vector<int> r;
    r.push_back(a.front());
    int last = a.front();
    for (auto x : a) {
        if (last != x) {
            r.push_back(x);
            last = x;
        }
    }
    return r;
}
```

5.4. 中位数

中位数到所有点的距离之和最小

5.5. 逆序数

```
int tmp[114514];
int merge(int a[], int l, int r) {  // 左闭右开
    int mid = (l + r) / 2;
    int cnt = 0;
    int i = l, j = mid;
    int k = l;
    while (i < mid && j < r) {
        if (a[i] <= a[j]) {
            tmp[k++] = a[i++];
        }
        else {
            tmp[k++] = a[j++];
            cnt += mid - i;
        }
    }
    while (i < mid) {
        tmp[k++] = a[i++];
    }
    while (j < r) {
        tmp[k++] = a[j++];
    }
    for (k = l; k < r; ++k) {
        a[k] = tmp[k];
    }
    return cnt;
}
int merge_sort(int a[], int l, int r) {
    if (r - l <= 1) {
        return 0;
    }
    int cnt = 0;
    int mid = (l + r) / 2;
    cnt += merge_sort(a, l, mid);
    cnt += merge_sort(a, mid, r);
    cnt += merge(a, l, r);
    return cnt;
}
```

6. 倍增

6.1. 倍增分段

```
bool check(int l, int r) {
    return true;
}
void beizeng(vector<int> a) {
    int n = a.size();

    for (int l = 0, r = 0; l < n;) {
        int p = 1;


        while (p) {
            // 左闭右开
            if (r + p <= n && check(l, r + p)) {
                r += p;
                p <<= 1;
            }
            else {
                p >>= 1;
            }
        }


        (l, r);
        l = r;
    }
```

```
}
```

6.2. ST 算法

```
/**
 * @brief ST 算法
 * @details f[i][j]记录[i,i+2^j)内的最大值
 * @details 先进行 O(nlogn)的预处理，之后可以进行 O(1)的在线回答
 */
const int N    = 1e5 + 5;
const int LOGN = 20;
int n;
int a[N];
int f[N][LOGN];
void st_prework() {
    for (int i = 1; i <= n; ++i) {
        f[i][0] = a[i];
    }
    int t = log2(n) + 1;
    for (int j = 1; j < t; ++j) {
        for (int i = 1; i < n - (1 << j); ++i) {
            f[i][j] = max(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);
        }
    }
}
int st_query(int l, int r) {
    int k = log2(r - l + 1);
    return max(f[l][k], f[r - (1 << k) + 1][k]);
}
```

7. 双指针

7.1. 相向指针

7.1.1. 接雨水问题

7.1.2. 最大矩形

7.2. 快慢指针

7.2.1. 环问题

7.3. 滑动窗口

7.3.1. 最小覆盖字串

7.3.2. 无重复字符的最长字串

7.4. 分离指针

7.4.1. 归并排序

8. 进阶数据结构

8.1. 并查集

```
struct DSU {
    vector<size_t> parent, size;
    DSU(size_t n)
        : parent(n),
          size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    size_t find_root(size_t x) {
        return parent[x] == x
                    ? x
                    : parent[x] = find_root(parent[x]);
    }
    void unite(size_t x, size_t y) {
        x = find_root(x), y = find_root(y);
        if (x == y) {
            return;
        }
        if (size[x] < size[y]) {
            swap(x, y);
        }
        parent[y] = x;
        size[x] += size[y];
    }
};
```

8.2. 线段树

```
int a[N];
int d[4 * N];
int lazy[4 * N];
struct Interval {
    int left, right;  // 所管理的区间
    int root;         // 对应的节点
};

#define MID            interval.left + (interval.right - interval.left) / 2
#define LEFT_INTERVAL  {interval.left, mid, interval.root * 2}
#define RIGHT_INTERVAL {mid + 1, interval.right, interval.root * 2 + 1}
void build(Interval interval) {
    if (interval.left == interval.right) {
        d[interval.root] = a[interval.left];
        return;
    }
    int mid = MID;
    build(LEFT_INTERVAL);
    build(RIGHT_INTERVAL);
    // 求和
    d[interval.root] = d[interval.root * 2] + d[interval.root * 2 + 1];
}


void push_down(Interval interval) {
    if (lazy[interval.root] && interval.left < interval.right) {
        int mid = MID;
        d[interval.root * 2] += lazy[interval.root] * (mid - interval.left + 1);
        d[interval.root * 2 + 1] += lazy[interval.root] * (interval.right - mid);
        lazy[interval.root * 2] += lazy[interval.root];
        lazy[interval.root * 2 + 1] += lazy[interval.root];
        lazy[interval.root] = 0;
    }
}
int query_sum(int find_left, int find_right, Interval interval) {
    if (find_left <= interval.left && interval.right <= find_right) {
        return d[interval.root];
    }
    push_down(interval);


    int mid = MID;
    int sum = 0;
    if (find_left <= mid) {
        sum += query_sum(find_left, find_right, LEFT_INTERVAL);
    }
    if (find_right > mid) {
        sum += query_sum(find_left, find_right, RIGHT_INTERVAL);
    }
    return sum;
}
void update(int update_left, int update_right, int delta_val, Interval interval) {
    if (update_left <= interval.left && interval.right <= update_right) {
        d[interval.root] += (interval.right - interval.left + 1) * delta_val;
        lazy[interval.root] += delta_val;
        return;
    }
    push_down(interval);


    int mid = MID;
    if (update_left <= mid) {
```

```
        update(update_left, update_right, delta_val, LEFT_INTERVAL);
    }
    if (update_right > mid) {
        update(update_left, update_right, delta_val, RIGHT_INTERVAL);
    }
    d[interval.root] = d[interval.root * 2] + d[interval.root * 2 + 1];
}
signed main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
    }
    build({1, n, 1});
    int find_left, find_right;
    cin >> find_left >> find_right;
    cout << query_sum(find_left, find_right, {1, n, 1});
    return 0;
}
```

9. 数论

9.1. 质数

9.1.1. 质数的判定

```
/**
 * @brief 试除法
 */
bool is_prime(int n) {
    if (n < 2) {
        return false;
    }
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}
```

9.1.2. 质数筛法

```
/**
 * @brief 埃式筛
 * @details O(n loglogn)
 */
```

```cpp
const int N = 10000;
bitset<N> v;
vector<int> primes;
void get_primes_1(int n) {
    v.reset();

    for (int i = 2; i <= n; ++i) {
        if (v[i]) {
            continue;
        }
        primes.push_back(i);
        for (int j = i * i; j <= n; j += i) {
            v[j] = true;
        }
    }
}
/**
 * @brief 线性筛
 * @details O(n)
 */
int u[N];           // 记录最小的质因子
vector<int> primes;  // 质数列表
void get_primes_2(int n) {
    memset(u, 0, sizeof(u));
    for (int i = 2; i <= n; ++i) {
        if (u[i] == 0) {
            u[i] = i;
            primes.push_back(i);
        }
        for (auto x : primes) {
            if (x > u[i] || x > n / i) {
                break;
            }
            u[i * x] = x;
        }
    }
}
```

9.1.3. 质因数分解

```cpp
struct Prime {
    int p, c;
};
vector<Prime> p;
void divide(int n) {
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            int cnt = 0;
            while (n % i == 0) {
                n /= i;
                cnt++;
            }
            p.push_back({i, cnt});
        }
    }
    if (n > 1) {
        p.push_back({n, 1});
    }
}
```

9.2. gcd

```cpp
/**
 * @name gcd
 */
int gcd(int a, int b) {
    return b ? gcd(b, a % b) : a;
}
/**
 * @name lcm
 */
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}
```

9.3. 因数

10. 动态规划

10.1. 背包

10.1.1. 01 背包

```cpp
/**
 * @brief 01 背包
 */
struct item {
    int w, v;
};
item a[N];
// int dp01[N][W];
int dp01[W];
void beibao01() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v;
    }

    // 法一：二维数组
    // for (int i = 1; i <= n; ++i) {
    //     for (int j = 0; j < a[i].w; ++j) {
    //         dp01[i][j] = dp01[i - 1][j];
    //     }
    //     for (int j = a[i].w; j <= w; ++j) {
    //         dp01[i][j] =
```

```
    //             max(dp01[i - 1][j],
    //                 dp01[i - 1][j - a[i].w] + a[i].v);
    //     }
    // }
    // cout << dp01[n][w];


    // 法二：一维数组（注意遍历方向）
    for (int i = 1; i <= n; ++i) {
        for (int j = w; j >= a[i].w; --j) {
            dp01[j] = max(dp01[j], dp01[j - a[i].w] + a[i].v);
        }
    }
    cout << dp01[w];
}
```

10.1.2. 多重背包

```
/**
 * @brief 多重背包
 */
struct item {
    int w, v;
    int c;
};
item a[N];

int dpdc[W];
void beibaodc() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v >> a[i].c;
    }


    // 法一：当成 01 背包，物品复制 c 份
    // for (int i = 1; i <= n; ++i) {
    //     // 小剪枝：如果足够大，可以直接当成完全背包
    //     // begin
    //     if (a[i].c * a[i].w >= w) {
    //         for (int j = a[i].w; j <= w; ++j) {
    //             dpdc[j] =
    //                 max(dpdc[j], dpdc[j - a[i].w] +
    //                 a[i].v);
    //         }
    //         continue;
    //     }
    //     // end
    //     while (a[i].c--)
    //         for (int j = w; j >= a[i].w; --j) {
    //             dpdc[j] =
    //                 max(dpdc[j], dpdc[j - a[i].w] +
    //                 a[i].v);
    //         }
    // }


    // 法二：二进制优化
    for (int i = 1; i <= n; ++i) {
        // 小剪枝：如果足够大，可以直接当成完全背包
        // begin
        if (a[i].c * a[i].w >= w) {
            for (int j = a[i].w; j <= w; ++j) {
```

```
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
            }
            continue;
        }
        // end

        // 二进制优化
        int leiji = 0;
        for (int k = 1; leiji + k <= a[i].c; leiji += k, k <<= 1) {
            for (int j = w; j >= a[i].w * k; --j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * k] + a[i].v * k);
            };
        }
        int r = a[i].c - leiji;
        if (r) {
            for (int j = w; j >= a[i].w * r; --j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * r] + a[i].v * r);
            }
        }
    }


    cout << dpdc[w];
}
```

10.1.3. 完全背包

```
/**
 * @brief 完全背包
 */
struct item {
    int w, v;
};
item a[N];


int dpwq[W];
void beibaowq() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v;
    }


    for (int i = 1; i <= n; ++i) {
        for (int j = a[i].w; j <= w; ++j) {
            dpwq[j] = max(dpwq[j], dpwq[j - a[i].w] + a[i].v);
        }
    }
    cout << dpwq[w];
}
```

10.1.4. 分组背包

11. 图论

11.1. 存储结构

11.1.1. 链式前向星

```cpp
/**
 * @name 链式前向星
 */
const int N = 1e5 + 5;
int n;
struct Edge {
    int next;    // 下一个坐标
    int to;      // 指向的节点
    int weight;  // 边权重
};
vector<Edge> edge;
vector<int> head(N, -1);  // 头节点的初始坐标
struct Edge {
    int next;
    int to;
    int weight;
};
vector<Edge> edge;

vector<int> head(N, -1);


void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}
void add_edge(int u, int v, int w) {
    edge.push_back({head[u], v, w});
    head[u] = edge.size() - 1;


    isnt_root[v] = true;
}
bool find_edge(int u, int v) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        if (edge[i].to == v) {
            return true;
        }
    }
    return false;
}


/**
 * @brief dfs 遍历
 * @details time O(n)
 */
// 有环图
vector<bool> visited;
void dfs(int u) {
    if (visited[u]) {
        return;
    }
    visited[u] = true;
    for (int i = head[u]; ~i; i = edge[i].next) {
        dfs(edge[i].to);
```

```
    }
}
// 无环图
void tree_dfs(int u, int fa) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == fa) {
            continue;
        }
        dfs(edge[i].to);
    }
}


/**
 * @brief 寻找根节点
 */
// 如果节点的情况未知
vector<int> head_list;  // 头节点的集合
vector<int> find_root() {
    vector<int> in_degree(head.size(), 0);


    for (auto e : edge) {
        ++in_degree[e.to];
    }


    vector<int> root_list;
    for (int x : head_list) {
        if (in_degree[x] == 0) {
            root_list.push_back(x);
        }
    }
    return root_list;
}
// 如果节点的情况已知
vector<bool> isnt_root;
void do_root() {
    for (int u = 1; u <= n; ++u) {
        if (!isnt_root[u]) {
            dfs(u);
        }
    }
}
```

11.1.2. 邻接链表

```
vector<int> edge[N];
vector<bool> visited(N);
void dfs(int u) {
    if (visited[u]) {
        return;
    }
    visited[u] = true;
    for (auto v : edge[u]) {
        dfs(v);
    }
}
```

11.2. 最短路

11.2.1. Dijkstra

```cpp
/**
 * @brief Dijkstra
 * @details time O(m log n)
 * @details 只允许非负权图
 */
const int N   = 1e3;
const int INF = 0x3f3f3f3f;
int n;
struct Edge {
    int v, w;
};
struct Node {
    int u;
    int dis;
    bool operator>(const Node& a) {
        return dis > a.dis;
    }
};
vector<Edge> edge[N];
int dis[N];
bool vis[N];
priority_queue<Node, vector<Node>, greater<Node>> pq;

void dijkstra(int s) {
    memset(dis, 0x3f, (n + 1) * sizeof(dis[0]));


    dis[s] = 0;
    pq.push({s, 0});
    while (!pq.empty()) {
        int u = pq.top().u;
        pq.pop();
        if (vis[u]) {
            continue;
        }
        vis[u] = true;
        for (auto e : edge[u]) {
            int v = e.v, w = e.w;
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                pq.push({v, dis[v]});
            }
        }
    }
}
```

11.2.2. SPFA

```cpp
/**
 * @name SPFA
 * @details time<=O(nm) space O(n)
 * @details 适用于正负边权的图，可以判断无最短路的情况
 */
const int N   = 1e3;
const int INF = 0x3f3f3f3f;
struct Edge {
    int v, w;
};
int n;
vector<Edge> edge[N];
int dis[N];
```

```cpp
int cnt[N];
bool vis[N];

queue<int> q;
bool SPFA(int s) {
    memset(dis, 0x3f, (n + 1) * (sizeof(dis[0])));


    dis[s] = 0;
    vis[s] = true;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();


        vis[u] = false;


        for (auto e : edge[u]) {
            int v = e.v, w = e.w;
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= n) {
                    return true;
                }
                if (!vis[v]) {
                    q.push(v);
                    vis[v] = true;
                }
            }
        }
    }
    return false;
}
```

11.3. 树

11.3.1. 深度、2^k 辈父节点

```cpp
/**
 * @brief depth 和 father
 * @param d[x]:表示 x 深度
 * @param f[x][k]:表示 x 的 2^k 辈祖先
 * @details time O(n)
 */

int d[N];
int f[N][LOGN];


void prework(int root) {
    queue<int> q;
    q.push(root);
    d[root] = 1;
    int t   = (int)(log(n) / log(2)) + 1;


    while (!q.empty()) {
        int u = q.front();
        q.pop();


        for (int i = head[u]; ~i; i = edge[i].next) {
```

```
            int v = edge[i].to;
            int w = edge[i].weight;
            if (d[v]) {
                continue;
            }
            q.push(v);


            d[v] = d[u] + 1;


            f[v][0] = u;
            for (int j = 1; j <= t; ++j) {
                f[v][j] = f[f[v][j - 1]][j - 1];
            }
        }
    }
}
```

11.3.2. 树的中心

```
/**
 * @brief  树的中心
 * @property 1.在直径上
 * @property 2.中心不唯一，但最多有 2 个，且两个中心相邻
 * @property 3.所有点到最远点的路径一定交汇于树的中心
 * @property 4.当中心为根节点时，到达直径两端的两条链一定分别是最长链和次长链
 * @property 5.树的中心到其他节点距离不超过直径的一半
 * @details 求法：寻找一个点 x，使其作为根节点时的最长链长度最短
 * @details time O(n)
 */
int d1[N];  // 以 1 为节点时 x 子树的最长链
int d2[N];  // 以 1 为节点时 x 子树的次长链
int up[N];  // x 子树外的最长链，必然经过 x 的父节点
// 如果找到点 x 使得 max(d1[x],up[x])最小，则 x 即为树的中心
void dfs_down(int u, int father) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == father) {
            continue;
        }
        dfs_down(v, u);

        if (d1[v] + w > d1[u]) {
            d2[u] = d1[u];
            d1[u] = d1[v] + w;
        }
        else if (d1[v] + w > d2[u]) {
            d2[u] = d1[v] + w;
        }
    }
}
void dfs_up(int u, int father) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == father) {
            continue;
        }
        up[v] = up[u] + w;
        if (d1[v] + w != d1[u]) {
            up[v] = max(up[v], d1[u] + w);
        }
```

```
        else {
            up[v] = max(up[v], d2[u] + w);
        }
        dfs_up(v, u);
    }
}
int get_center() {
    dfs_down(1, 0);
    dfs_up(1, 0);
    int minlen = INT_MAX;
    int ans    = 0;
    int ans2   = 0;
    for (int i = 1; i <= n; ++i) {
        // 其中一个中心
        if (max(d1[i], up[i]) < minlen) {
            minlen = max(d1[i], up[i]);
            ans    = i;
            ans2   = 0;
        }
        // 另一个中心
        else if (max(d1[i], up[i]) == minlen) {
            ans2 = i;
        }
    }
    return ans;
}
```

11.3.3. LCA

```
/**
 * @brief 树的直径
 * @details 树形 dp
 * @param d[x]: 以 x 节点为根节点，向下延申可形成的最长距离
 * @param ans: 经过 x 节点形成的最长直径
 */
int d[N];
int ans;
void dp(int u, int fa) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;

        if (v == fa) {
            continue;
        }
        dp(v, u);

        // 这里的 d[u]为 1-(i-1)内的最大向下距离
        ans  = max(ans, d[u] + d[v] + w);
        d[u] = max(d[u], d[v] + w);
    }
}

/**
 * @brief LCA
 * @details 树上倍增
 * @details 预处理 O(nlogn)，询问 O(logn)
 * @param f[x][k]:表示 x 的 2^k 辈祖先
 * @param d[x]:表示 x 深度
 */

int f[N][LOGN];
```

```
int d[N];


void prework(int root) {
    queue<int> q;
    q.push(root);
    d[root] = 1;
    int t   = (int)(log(n) / log(2)) + 1;


    while (!q.empty()) {
        int u = q.front();
        q.pop();


        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            if (d[v]) {
                continue;
            }
            q.push(v);


            d[v] = d[u] + 1;


            f[v][0] = u;
            for (int j = 1; j <= t; ++j) {
                f[v][j] = f[f[v][j - 1]][j - 1];
            }
        }
    }
}
int lca(int x, int y) {
    if (d[x] > d[y]) {
        swap(x, y);
    }
    int t = (int)(log(n) / log(2)) + 1;


    for (int i = t; i >= 0; --i) {
        if (d[f[y][i]] >= d[x]) {
            y = f[y][i];
        }
    }
    if (x == y) {
        return x;
    }
    for (int i = t; i >= 0; --i) {
        if (f[x][i] != f[y][i]) {
            x = f[x][i];
            y = f[y][i];
        }
    }
    return f[x][0];
}
```

11.3.4. 树的重心

```
int size[N];
int now_min;  // 去掉重心后的最大子树大小
int ans;
void dfs(int u) {
    if (visited[u]) {
        return;
```

```
    }
    visited[u] = true;

    size[u] = 1;

    int max_part = 0;  // 表示删掉 x 后分成的最大子树的大小

    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        dfs(v);

        size[u] += size[v];

        max_part = max(max_part, size[v]);
    }
    max_part = max(max_part, n - size[u]);
    if (max_part < now_min) {
        now_min = max_part;
        ans     = u;
    }
}
```

11.4. 生成树

11.4.1. 最小生成树

11.4.1.1. Kruskal 算法

```
/**
 * @brief Kruskal 算法
 * @details time O(m log m)
 */
struct DSU {
    vector<size_t> parent, size;
    DSU(size_t n)
        : parent(n),
          size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    size_t find_root(size_t x) {
        return parent[x] == x ? x : parent[x] = find_root(parent[x]);
    }
    void unite(size_t x, size_t y) {
        x = find_root(x), y = find_root(y);
        if (x == y) {
            return;
        }
        if (size[x] < size[y]) {
            swap(x, y);
        }
        parent[y] = x;
        size[x] += size[y];
    }
};


struct Temp_edge {
```

```
    int u, v, w;
};
bool operator<(const Temp_edge& a, const Temp_edge& b) {
    return a.w < b.w;
}
void kruskal(vector<Temp_edge>& lists) {
    int n = lists.size();
    sort(lists.begin(), lists.end());


    DSU dsu(n);
    for (auto [u, v, w] : lists) {
        int x = dsu.find_root(u);
        int y = dsu.find_root(v);
        if (x == y) {
            continue;
        }
        dsu.unite(x, y);


        // add_edge(u, v, w);
    }
}
```

11.4.1.2. Prim 算法

```
/**
 * @brief Prim 算法
 * @details O(m log n)
 */
const int N = 1e5 + 5;
int dis[N];  // 到当前生成树的最小距离
```

```
// pii=<dis,u>
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> q;
vector<bool> visited;
int prim(int root) {
    visited.resize(n, false);
    memset(dis, 0x3f, sizeof(dis));
    dis[root] = 0;
    q.push({0, root});
    int cnt = 0;
    while (!q.empty()) {

        auto [d, u] = q.top();
        q.pop();


        if (visited[u]) {
            continue;
        }
        visited[u] = true;


        ++cnt;


        // do_something(father,u,d);


        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;


            if (w < dis[v]) {
                dis[v] = w;
                q.push({w, v});
```

```
            }
        }


        if (cnt >= n) {
            break;
        }
    }
    if (cnt < n) {
        // 不连通
        return -1;
    }
    return 0;
}
```

11.5. 二分图

11.5.1. 染色判定法

```
/**
 * @brief 二分图的染色判定法
 */
int color[N];
bool dfs_color(int u, int c) {
    color[u] = c;
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (color[v] == 0) {
            if (!dfs_color(v, 3 - c)) {
                return false;
            }
        }
        else if (color[v] == c) {
            return false;
        }
    }
    return true;
}
bool judge() {
    for (int i = 1; i <= n; ++i) {
        if (color[i] == 0) {
            if (!dfs_color(i, 1)) {
                return false;
            }
        }
    }
    return true;
}
```

11.5.2. 最大匹配数(匈牙利算法)

```
/**
 * @brief 二分图的最大匹配
 * @brief 匈牙利算法
 * @details time O(n m)
 * @return 最大匹配数
 */
int match[N];
int visited[N];
bool dfs(int u) {
```

```
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;

        if (!visited[v]) {
            visited[v] = true;
            if (!match[v] || dfs(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}
int Hungarian() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            ans++;
        }
    }
    return ans;
}
```

12. 字符串

12.1. KMP 模式匹配

```
string s;
string k;
vector<int> kmp;
void set_kmp(void) {
    kmp[0] = 0;
    for (int i = 1; i <= k.size(); ++i) {
        int j = kmp[i - 1];
        while (j > 0 && k[i] != k[j]) {
            j = kmp[j - 1];
        }
        if (k[i] == k[j]) {
            ++j;
        }
        kmp[i] = j;
    }
}
int main() {
    cin >> s >> k;
    kmp.resize(k.size());
    set_kmp();
    for (int i = 0, j = 0; i < s.size(); ++i) {
        while (j > 0 && s[i] != k[j]) {
            j = kmp[j - 1];
        }
        if (s[i] == k[j]) {
            ++j;
```

```
        }
        if (j == k.size()) {
            cout << i - k.size() + 2 << endl;
            j = kmp[j - 1];
        }
    }
    for (int i = 0; i < kmp.size(); ++i) {
        cout << kmp[i] << " ";
    }
    return 0;
}
```

12.2. Trie 字典树

```
struct trie {
    int next[100000][26];
    int cnt;
    bool exist[100000];

    void insert(const char* s, int len) {
        int ptr = 0;
        for (int i = 0; i < len; ++i) {
            int c = s[i] - 'a';
            if (!next[ptr][c]) {
                ++cnt;
                next[ptr][c] = cnt;
            }
            ptr = next[ptr][c];
        }
        exist[ptr] = true;
    }
    bool find(const char* s, int len) {
        int ptr = 0;
        for (int i = 0; i < len; ++i) {
            int c = s[i] - 'a';
            if (!next[ptr][c]) {
                return false;
            }
            ptr = next[ptr][c];
        }
        return exist[ptr];
    }
};
```

12.3. SA

```
string s;
int sa[N << 1];
int rk[N << 1];

// O(n^2 log n)
void get_sa1(void) {
    iota(sa, sa + s.size(), 0);
    sort(sa, sa + s.size(), [&](int i, int j) {
        return s.substr(i) < s.substr(j);
    });
}
void get_rk_from_sa(void) {
    for (int i = 0; i < s.size(); ++i) {
        rk[sa[i]] = i;
    }
}
```

```cpp
// O(n log^2 n)
int oldrk[N << 1];
void get_sa2(void) {
    // 第一层排序
    for (int i = 0; i < s.size(); ++i) {
        sa[i] = i;
        rk[i] = s[i];
    }
    // 二层及以上排序
    for (int w = 1; w < s.size(); w <<= 1) {
        sort(sa, sa + s.size(), [&](int i, int j) {
            // 这里需要两倍空间，否则就得老老实实防止越界
            return rk[i] == rk[j] ? rk[i + w] < rk[j + w] : rk[i] < rk[j];
        });
        for (int i = 0; i < s.size(); ++i) {
            oldrk[i] = rk[i];
        }
        // 双指针去重
        rk[sa[0]] = 0;
        for (int i = 1, j = 0; i < s.size(); ++i) {
            if (oldrk[sa[i]] == oldrk[sa[i - 1]] && oldrk[sa[i] + w] == oldrk[sa[i - 1] +
w]) {
                rk[sa[i]] = j;
            }
            else {
                rk[sa[i]] = ++j;
            }
        }
    }
}
```

12.4. SAM

```cpp
struct state {
    int len;
    int link;
    map<char, int> next;
};
const int N = 1000000;
state st[N * 2];
int sz, last;
void sam_init() {
    st[0].len  = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}
void sam_extend(char c) {
    int cur     = sz++;
    st[cur].len = st[last].len + 1;
    int p       = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p             = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    }
    else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        }
```

```
        else {
            int clone      = sz++;
            st[clone].len  = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p             = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
void sam_build(string s) {
    sam_init();
    for (auto x : s) {
        sam_extend(x);
    }
}
```